

实验四：基于 Transformer 的中英机器翻译

朱立景 2025E8013282177

一、概述

本实验旨在构建一个端到端的中英机器翻译系统。任务为将中文句子翻译为英文，采用 Encoder-Decoder 架构的 Transformer 模型，在 NiuTrans 提供的开源中英平行语料库上进行训练，并以 BLEU-4 作为核心评估指标。

任务定义： 给定中文源句 $x = (x_1, x_2, \dots, x_m)$ ，模型学习生成对应的英文目标句 $y = (y_1, y_2, \dots, y_n)$ ，使翻译结果在 n-gram 层面尽可能接近参考译文。

数据集： NiuTrans 开源中英平行语料，包含 10 万条训练句对、400 条开发句对和 1000 条测试句。

解决方案： 基于 PyTorch 实现标准 Transformer 序列到序列模型，配合 Noam 学习率预热调度、Label Smoothing 交叉熵损失和贪心解码策略，在 GPU 上完成训练与评估。

实验环境： Conda, Python 3.10, PyTorch 2.5.1 + CUDA, NVIDIA A800 GPU。

二、解决方案

2.1 整体流程

1 | 中文句子 → 分词 → 词表编码 → Encoder → Decoder → 线性输出层 → 英文词序列 → 解码为文本

数据预处理阶段对中英文分别构建词表（含 `<pad>`、`<unk>`、`<bos>`、`<eos>` 特殊符号），训练时采用 Teacher Forcing，推理时使用贪心解码逐词生成目标句。

2.2 网络结构设计

模型 `Seq2SeqTransformer` 遵循 Vaswani 等人提出的 Transformer 架构，主要包含以下模块：

模块	说明
Token Embedding	中英文各自独立的词嵌入层，输出乘以 $\sqrt{d_{model}}$ 进行缩放
Positional Encoding	正弦/余弦位置编码，为序列注入位置信息
Transformer Encoder	3 层，8 头多头自注意力，隐藏维度 256
Transformer Decoder	3 层，含掩码自注意力与交叉注意力
前馈网络 (FFN)	维度 512，激活函数 ReLU
Generator	线性层，将解码器输出映射到目标词表大小

超参数配置：

参数	取值
<code>emb_size</code>	256
<code>nhead</code>	8
<code>num_encoder_layers</code>	3
<code>num_decoder_layers</code>	3
<code>ffn_hid_dim</code>	512
<code>dropout</code>	0.1
<code>max_decode_len</code>	128

核心网络结构代码如下：

```

1  class Seq2SeqTransformer(nn.Module):
2      def __init__(self, num_encoder_layers, num_decoder_layers, emb_size, nhead,
3                    src_vocab_size, tgt_vocab_size, dim_feedforward=512, dropout=0.1):
4          super().__init__()
5          self.transformer = nn.Transformer(
6              d_model=emb_size, nhead=nhead,
7              num_encoder_layers=num_encoder_layers,
8              num_decoder_layers=num_decoder_layers,
9              dim_feedforward=dim_feedforward, dropout=dropout,
10             )
11         self.generator = nn.Linear(emb_size, tgt_vocab_size)
12         self.src_tok_emb = TokenEmbedding(src_vocab_size, emb_size)
13         self.tgt_tok_emb = TokenEmbedding(tgt_vocab_size, emb_size)
14         self.positional_encoding = PositionalEncoding(emb_size, dropout=dropout)
15
16         def forward(self, src, tgt, src_mask, tgt_mask,
17                     src_padding_mask, tgt_padding_mask, memory_key_padding_mask):
18             src_emb = self.positional_encoding(self.src_tok_emb(src))
19             tgt_emb = self.positional_encoding(self.tgt_tok_emb(tgt))
20             outs = self.transformer(src_emb, tgt_emb, src_mask=src_mask, tgt_mask=tgt_mask,
21                                   src_key_padding_mask=src_padding_mask,
22                                   tgt_key_padding_mask=tgt_padding_mask,
23                                   memory_key_padding_mask=memory_key_padding_mask)
24             return self.generator(outs)

```

位置编码采用经典的正弦/余弦方案：

```

1  class PositionalEncoding(nn.Module):
2      def __init__(self, emb_size, dropout=0.1, max_len=5000):
3          super().__init__()
4          den = torch.exp(-torch.arange(0, emb_size, 2) * math.log(10000) / emb_size)
5          pos = torch.arange(0, max_len).reshape(max_len, 1)
6          pos_embedding = torch.zeros((max_len, emb_size))
7          pos_embedding[:, 0::2] = torch.sin(pos * den)
8          pos_embedding[:, 1::2] = torch.cos(pos * den)
9          self.dropout = nn.Dropout(dropout)
10         self.register_buffer("pos_embedding", pos_embedding.unsqueeze(1))

```

训练时对目标序列使用因果掩码（Causal Mask），防止解码器看到未来信息；同时对 padding 位置使用 key padding mask 加以屏蔽。

2.3 损失函数设计

采用带 **Label Smoothing** 的交叉熵损失。标准 one-hot 标签会使模型过度自信，Label Smoothing 将真实标签的概率从 1.0 软化为 $1 - \epsilon$ ，其余概率均匀分配给词表中其他 token ($\epsilon = 0.1$)，有助于提升泛化能力。

```
1 loss_fn = nn.CrossEntropyLoss(  
2     ignore_index=tgt_vocab.stoi[PAD_TOKEN],  
3     label_smoothing=0.1,  
4 )
```

训练时采用 Teacher Forcing：解码器输入为目标序列左移一位（`tgt[:-1]`），预测目标为右移一位（`tgt[1:]`），对 padding 位置不计入损失。

2.4 优化器设计

优化器选用 **Adam** ($\beta_1 = 0.9$, $\beta_2 = 0.98$, $\epsilon = 10^{-9}$)，学习率由 **Noam 调度策略** 动态控制：

$$\text{lr} = d_{\text{model}}^{-0.5} \cdot \min(\text{step}^{-0.5}, \text{step} \cdot \text{warmup_steps}^{-1.5})$$

该策略在训练初期线性增大学习率（warmup），之后按步数平方根衰减，是 Transformer 原论文的标准做法。`warmup_steps` 设为 4000，Adam 基础学习率设为 1.0，由调度器按公式缩放。

```
1 def build_noam_lr_lambda(model_size, warmup_steps, lr_factor=1.0):  
2     def noam_lambda(step):  
3         step = max(1, step)  
4         return lr_factor * (model_size ** -0.5) * min(  
5             step ** -0.5, step * (warmup_steps ** -1.5)  
6         )  
7     return noam_lambda  
8  
9 optimizer = torch.optim.Adam(model.parameters(), lr=1.0, betas=(0.9, 0.98), eps=1e-9)  
10 scheduler = torch.optim.lr_scheduler.LambdaLR(  
11     optimizer, lr_lambda=build_noam_lr_lambda(model_size=256, warmup_steps=4000, lr_factor=1.0)  
12 )
```

此外，每个 batch 训练后执行梯度裁剪（`clip_grad_norm_`，阈值 1.0），防止梯度爆炸。

2.5 解码策略

推理阶段采用 **贪心解码（Greedy Decoding）**：每步选取概率最高的 token 作为输出，直至生成 `<eos>` 或达到最大长度 128。编码器对源句编码一次，解码器逐步自回归生成目标句。

```

1  @torch.no_grad()
2  def greedy_decode(model, src, src_mask, max_len, start_symbol, eos_symbol, device):
3      memory = model.encode(src, src_mask)
4      ys = torch.ones(1, 1, dtype=torch.long, device=device) * start_symbol
5      for _ in range(max_len - 1):
6          tgt_mask = generate_square_subsequent_mask(ys.size(0), device)
7          out = model.decode(ys, memory, tgt_mask)
8          prob = model.generator(out.transpose(0, 1)[: , -1])
9          _, next_word = torch.max(prob, dim=1)
10         ys = torch.cat([ys, torch.ones(1, 1, dtype=torch.long, device=device) * next_word.item()],
11                        dim=0)
12         if next_word.item() == eos_symbol:
13             break
14     return ys.flatten()

```

2.6 评估指标

BLEU-4 (Bilingual Evaluation Understudy) 衡量生成译文与参考译文在 1-gram 至 4-gram 上的加权几何平均精确率，并施加简短惩罚 (Brevity Penalty)。本实验在开发集上逐 epoch 计算 corpus-level BLEU-4，保存 BLEU 最高的 checkpoint。

三、实验分析

3.1 数据集介绍

本实验使用 NiuTrans 开源中英平行语料库，数据已预先完成分词处理（词与词之间以空格分隔）。

数据集	文件	规模	格式说明
训练集	chinese.txt + english.txt	100,000 句对	逐行对齐的中英平行句
开发集	Niu.dev.txt	400 句对	中文行、空行、英文行交替排列
测试集	Niu.test.txt	1,000 句	仅含中文源句，无参考译文

数据示例（训练集）：

- 中文：1998年，经过统一部署，伊犁州，地两级党委开始尝试以宣讲团的形式 ...
- 英文：in 1998 , the yili autonomous prefecture cpc committee and the yili prefecture cpc committee made unified arrangements ...

语料涵盖新闻、政治、经济等领域，句子长度差异较大。词表构建时过滤频次低于 2 的 token，最大词表大小 50,000，最终得到中文词表 32,697 词、英文词表 22,811 词。

3.2 实验设置

配置项	取值
训练轮数 (epochs)	10
批大小 (batch_size)	64
学习率缩放因子 (lr)	1.0
Warmup 步数	4000
Label Smoothing	0.1
随机种子	42
设备	CUDA (NVIDIA A800)

3.3 实验结果

训练过程各 epoch 的训练损失、开发集 BLEU-4 及当前学习率如下：

Epoch	Train Loss	Dev BLEU-4	Learning Rate
01	6.6226	4.07	3.86e-4
02	4.7674	10.18	7.72e-4
03	4.1450	14.39	9.13e-4
04	3.7734	15.10	7.90e-4
05	3.5423	16.75	7.07e-4
06	3.3879	19.11	6.45e-4
07	3.2743	19.69	5.98e-4
08	3.1858	19.10	5.59e-4
09	3.1155	19.94	5.27e-4
10	3.0545	20.28	5.00e-4

最佳开发集 **BLEU-4：20.28**（第 10 个 epoch）。测试集翻译结果输出至 `data/pred.test.en.txt`，最优模型保存为 `checkpoints/best_transformer_mt.pt`。最优模型在测试集的翻译输出到pred.test.en.txt中。

3.4 结果分析

(1) 训练损失持续下降

训练损失从第 1 轮的 6.62 稳步降至第 10 轮的 3.05，降幅约 54%，表明模型在有效拟合训练数据。损失曲线平滑，未出现剧烈震荡，说明 Noam 调度与梯度裁剪起到了稳定训练的作用。

(2) BLEU-4 快速提升后趋于收敛

- 前 3 个 epoch BLEU 从 4.07 快速升至 14.39，模型迅速学会基本的词序对齐与高频词翻译。
- 第 4–6 轮进入平台期后再次突破，BLEU 从 15.10 跃升至 19.11，可能与学习率衰减后模型开始精细调整有关。
- 第 7–10 轮 BLEU 在 19–20 之间波动，最终稳定在 20.28，显示模型已接近当前配置下的性能上限。

(3) 学习率调度效果

学习率在第 3 个 epoch 达到峰值约 $9.13e-4$ (warmup 结束)，之后按 Noam 公式逐步衰减至 $5.00e-4$ 。warmup 阶段避免了训练初期梯度过大导致的不稳定，衰减阶段则有助于模型收敛到更优解。

(4) 性能瓶颈分析

BLEU-4 约 20 分在 10 万规模语料、3 层 Transformer 的配置下属于合理水平，但仍有提升空间。主要限制因素包括：

- **模型规模较小**：3 层 Encoder/Decoder、256 维嵌入，参数量有限，对长句和低频词的建模能力不足。
- **贪心解码**：每步只取概率最高的词，无法探索多条候选路径，容易陷入局部最优。
- **无子词切分**：基于词级别的分词导致未登录词 (OOV) 映射为 `<unk>`，损失语义信息。
- **训练轮数有限**：第 8 轮 BLEU 出现小幅回落 (19.10)，第 9-10 轮才恢复，说明模型尚未完全收敛，增加 epoch 或使用早停策略可能有所帮助。

四、总结

本实验成功搭建了基于 Transformer 的中英机器翻译系统，在 NiuTrans 10 万句平行语料上训练 10 个 epoch，开发集 BLEU-4 达到 **20.28**。实验验证了 Transformer Encoder-Decoder 架构在神经机器翻译任务上的有效性，Noam 学习率预热与 Label Smoothing 对训练稳定性与泛化均有积极作用。

主要收获：

1. 完整实现了 Transformer 机器翻译的数据加载、模型训练、贪心解码与 BLEU-4 评估流程。
2. 掌握了序列到序列任务中掩码机制、Teacher Forcing、位置编码等关键技术。
3. 通过实验观察到了典型的"快速学习 → 平台期 → 缓慢收敛"训练曲线。